

MELIKA

*Plataforma de Gestión de Historias Clínicas
y Administración de Consultas Médicas*

MANUAL TÉCNICO

Documentación de Arquitectura, Instalación, Despliegue y Operación

Versión 1.0.0

Julio de 2026

Repositorio: github.com/Imjuaniss/proyecto-melika

Stack: React · Vite · CSS puro · Node.js · Express · PostgreSQL · Railwa

1. Introducción y Alcance

MELIKA es una plataforma web de gestión de historias clínicas y administración de consultas médicas, diseñada para conectar a pacientes, médicos y personal administrativo dentro de un mismo flujo digital. El sistema cubre el ciclo completo de la atención en salud a nivel de plataforma: agendamiento de citas, consulta clínica, registro de historias, emisión de fórmulas médicas y órdenes de examen, y generación de documentos clínicos en formato PDF.

Este manual técnico está dirigido al equipo de desarrollo y al personal técnico responsable de instalar, configurar, desplegar y mantener el sistema. Documenta la arquitectura real del proyecto, su modelo de datos, los contratos de la API REST, el proceso de despliegue en Railway y los procedimientos operativos de mantenimiento, con base directa en el código fuente del repositorio.

1.1 Objetivos del Documento

- Establecer los requisitos técnicos necesarios para ejecutar MELIKA en un entorno local de desarrollo.
- Documentar la arquitectura, el modelo de datos y los módulos funcionales del sistema.
- Describir el proceso de despliegue en Railway, incluyendo la configuración de ambos servicios (cliente y servidor).
- Servir como referencia de consulta para la resolución de incidentes técnicos frecuentes.
- Dejar constancia de las decisiones de arquitectura no evidentes a simple vista (orden de rutas, envío de correo vía API en lugar de SMTP, generación de PDF en el cliente, entre otras).

1.2 Audiencia

Este documento asume conocimientos previos de JavaScript, desarrollo web full-stack y fundamentos de bases de datos relacionales. Está pensado para desarrolladores que se incorporan al proyecto, para el propio equipo de mantenimiento y para cualquier persona técnica que deba operar el sistema en producción.

2. Arquitectura General del Sistema

MELIKA se organiza como un monorepo con dos aplicaciones independientes — client/ y server/ — que se despliegan como dos servicios separados en Railway y se comunican exclusivamente a través de una API REST sobre HTTPS. No existe renderizado del lado del servidor: el cliente es una Single Page Application (SPA) que consume la API del backend.

2.1 Stack Tecnológico

Capa	Tecnología	Rol en el proyecto
Frontend	React 19 + Vite 8	Interfaz de usuario tipo SPA, enrutada con react-router-dom v7
Estilos	CSS puro (BEM)	Sin frameworks de UI (Tailwind, Bootstrap, librerías de componentes) por decisión de proyecto
Documentos PDF	@react-pdf/renderer, @pdfslick/react	Generación y visualización de historias clínicas, fórmulas y órdenes en PDF
Backend	Node.js + Express 5	API REST, autenticación, lógica de negocio clínica y administrativa
Base de datos	PostgreSQL (pg.Pool)	Persistencia relacional con connection pooling
Autenticación	JWT + bcrypt	Tokens firmados con expiración y hashing de contraseñas
Correo transaccional	Gmail API (googleapis)	Envío de notificaciones vía HTTPS, evitando bloqueo de puertos SMTP en Railway
Hosting	Railway (PaaS)	Dos servicios independientes (cliente y servidor) + plugin PostgreSQL administrado

2.2 Vista Lógica de la Arquitectura

El siguiente esquema resume el flujo de una petición típica dentro del sistema:

```

Navegador (SPA React)
  | fetch() con header Authorization: Bearer <JWT>
  ▼
Servicio Railway "client" (Vite build servido con `serve`)
  | VITE_API_URL apunta al dominio público del servicio "server"
  ▼
Servicio Railway "server" (Express)
  | cors() valida el origen contra FRONTEND_URL
  | authMiddleware.verifyToken → decodifica el JWT
  | authMiddleware.isAdmin / isMedico / isPaciente → valida el rol
  ▼

```

```
Controladores (src/controllers/*.js)
  |  Lógica de negocio + validaciones clínicas
  ▼
pg.Pool (src/config/db.js)
  ▼
PostgreSQL (plugin administrado por Railway)
```

2.3 Principios de Diseño Aplicados

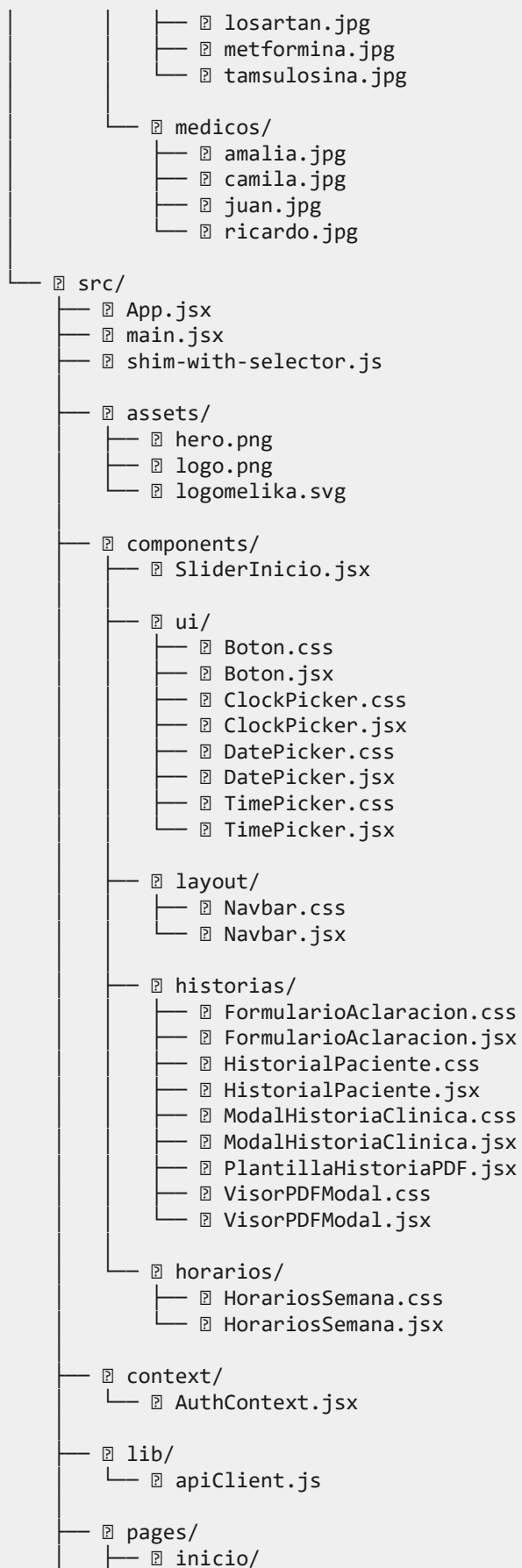
- Separación estricta cliente/servidor: dos repositorios lógicos (client/, server/) con ciclos de despliegue independientes.
- Autorización por rol a nivel de ruta: cada endpoint sensible se protege explícitamente con `verifyToken` y el middleware de rol correspondiente (`isAdmin`, `isMedico`, `isPaciente`), nunca de forma implícita.
- Inmutabilidad clínico-legal: una historia clínica principal nunca se sobrescribe; las correcciones se registran como nuevos documentos de tipo `nota aclaracion` o `nota evolucion`, preservando el historial completo (ver sección 4.3).
- Generación de documentos en el cliente: los PDF clínicos se generan en el navegador con `@react-pdf/renderer` a partir de los datos ya autorizados que devuelve la API, evitando duplicar esa lógica en el backend.

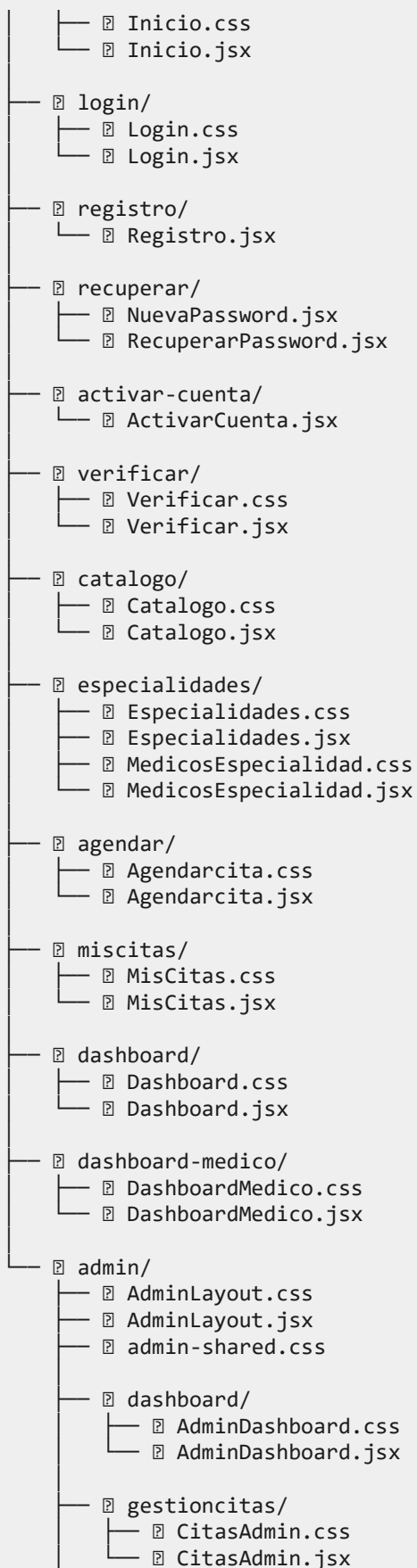
3. Estructura del Proyecto

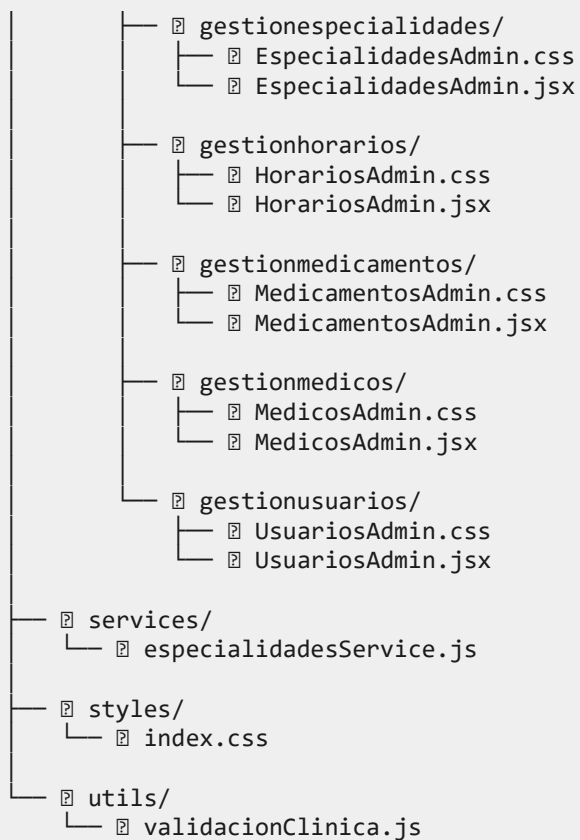
El repositorio se organiza en dos carpetas raíz independientes, cada una con su propio package.json, ciclo de instalación y despliegue.

3.1 Carpeta client/

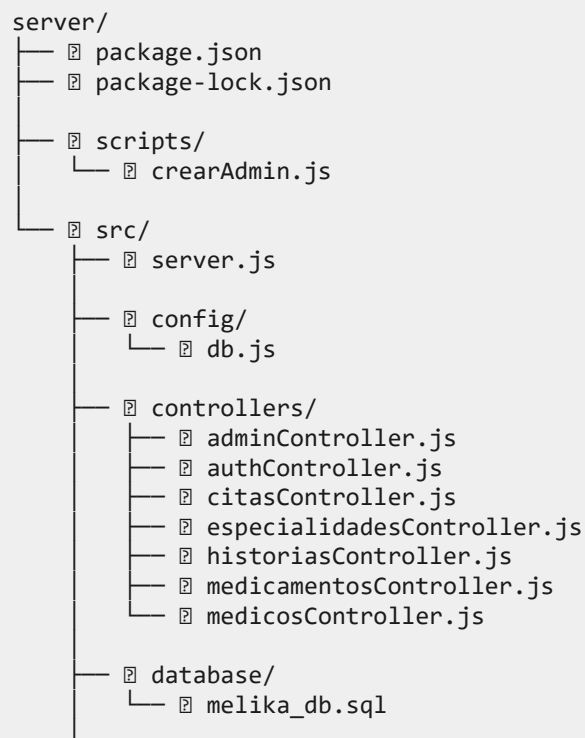
```
client/
├── package.json
├── package-lock.json
├── index.html
├── vite.config.js
├── eslint.config.js
├── railway.json
├── README.md
├── public/
│   ├── icons.svg
│   └── imagenes/
│       ├── Exámenes-3-plata.pdf
│       ├── Formula-3-plata.pdf
│       ├── Formula_Medica_1.pdf
│       ├── clinica-fachada.jpg
│       ├── consultorio.jpg
│       ├── equipo-medico.jpg
│       ├── especialidades/
│       │   ├── cardiologia.jpg
│       │   ├── dermatologia.jpg
│       │   ├── ginecologia.jpg
│       │   ├── medicina-general.jpg
│       │   ├── neurologia.jpg
│       │   ├── nutricion.jpg
│       │   ├── oftalmologia.jpg
│       │   ├── ortopedia.jpg
│       │   ├── otorrino.jpg
│       │   ├── pediatria.jpg
│       │   ├── psiquiatria.jpg
│       │   ├── urologia.jpg
│       │   └── veterinaria.jpg
│       └── medicamentos/
│           ├── acetaminofen-jarabe.jpg
│           ├── acido-folico.jpg
│           ├── acido-retinoico.jpg
│           ├── amoxicilina.jpg
│           ├── atorvastatina.jpg
│           ├── casa.jpg.jpg
│           ├── gabapentina.png
│           ├── ibuprofeno.jpg
│           └── images.jpg
```







3.2 Carpeta server/




```
├── middleware/
│   └── authMiddleware.js
├── routes/
│   ├── adminRoutes.js
│   ├── authRoutes.js
│   ├── citasRoutes.js
│   ├── especialidadesRoutes.js
│   ├── historiasRoutes.js
│   ├── medicamentosRoutes.js
│   └── medicosRoutes.js
├── services/
│   └── emailService.js
└── utils/
    ├── validacionesHistoria.js
    └── validacionesRegistro.js
```

3.3 Convención de Nomenclatura

Los nombres de carpetas y archivos del dominio clínico y administrativo se mantienen en español (historiasController.js, gestionmedicos/, EspecialidadesAdmin.jsx), consistente con el idioma del negocio y de los usuarios finales del sistema. Cada módulo del panel de administración replica la misma convención: una carpeta gestionX/ con un componente XAdmin.jsx y su hoja de estilos XAdmin.css.

4. Modelo de Datos

La base de datos de MELIKA es PostgreSQL. El esquema vive en `server/src/database/melika_db.sql` y se ha ido extendiendo mediante migraciones incrementales acumuladas en el mismo archivo, todas escritas con `ADD COLUMN IF NOT EXISTS / CREATE TABLE IF NOT EXISTS` para ser reejecutables sin riesgo.

4.1 Entidades Principales

Tabla	Propósito
usuarios	Cuenta única para pacientes, médicos y administradores. El rol se define en la columna rol (paciente / medico / admin) y se restringe con CHECK.
codigos_verificacion	Códigos de 6 dígitos de un solo uso para verificación de registro y recuperación de contraseña, con expiración.
tokens_invitacion	Tokens de invitación para activar cuentas de médico o administrador (flujo distinto al autorregistro de pacientes).
especialidades	Catálogo de especialidades médicas ofrecidas, con precio base e imagen.
medicos	Perfil profesional vinculado 1:1 a usuarios (id_usuario UNIQUE), con número de registro, tarifa, modalidades de atención y calificación.
frangas_horarias	Bloques de disponibilidad de cada médico (fecha, hora_inicio, hora_fin), con restricción chk_horas_orden.
citas	Tabla núcleo del sistema: vincula paciente, médico, especialidad y franja horaria; controla el estado del ciclo de vida de la cita.
historias_clinicas	Registro clínico completo de una consulta: anamnesis, examen físico, diagnóstico CIE-10, signos vitales, plan de tratamiento y cierre legal.
documentos_clinicos	Documentos generados o adjuntos (historia, fórmula, orden de examen, documento externo), en lógica append-only.
recetas_medicas	Detalle línea a línea de los medicamentos formulados en una historia clínica.
ordenes_examenes	Órdenes de laboratorio o imagenología asociadas a una historia clínica.
logs_citas	Auditoría de cambios sobre citas: snapshot JSONB del estado antes y después de cada operación.

4.2 Relaciones Clave

```

usuarios 1—1 medicos (id_usuario)
medicos  1—N frangas_horarias
medicos  1—N citas
usuarios 1—N citas          (paciente)

```

```

citas      1—1 historias_clinicas (historia_principal, ver 4.3)
historias_clinicas 1—N recetas_medicas
historias_clinicas 1—N ordenes_exámenes
historias_clinicas 1—N documentos_clinicos
especialidades 1—N medicos
especialidades 1—N medicamentos (vínculo opcional, id_especialidad)

```

4.3 Reglas de Integridad Clínica

El módulo de historias clínicas es el más sensible del sistema y concentra la mayor parte de la lógica de integridad de datos. Las siguientes reglas están implementadas a nivel de base de datos, no solo en el backend, para garantizar que ningún cliente de la API pueda saltárselas:

- Versionado legal: una historia_clinica puede ser de tipo historia_principal, nota_aclaracion o nota_evolucion. Solo puede existir una historia_principal por cita — se garantiza con un índice único parcial (uq_historia_principal_por_cita) en lugar de un UNIQUE simple, precisamente para permitir múltiples aclaraciones sobre la misma cita.
- Historia principal completa: un CHECK (chk_historia_principal_completa) obliga a que toda historia_principal tenga anamnesis, antecedentes patológicos, antecedentes alérgicos, examen físico, diagnóstico CIE-10 y su descripción, plan de tratamiento y firma del médico (nombre y RETHUS) antes de poder guardarse.
- Rangos clínicos válidos: constraints NOT VALID acotan los signos vitales a rangos fisiológicamente posibles (p. ej. frecuencia cardíaca entre 20 y 250, temperatura entre 30 °C y 43 °C), evitando errores de digitación sin bloquear datos históricos ya existentes.
- Formato CIE-10: el diagnóstico se valida con una expresión regular (letra + 2 dígitos + subcategoría opcional).
- Cierre legal obligatorio: nombre y número RETHUS del médico son obligatorios en toda historia y en toda nota, sin excepción.
- Inmutabilidad de documentos clínicos: la tabla documentos_clinicos no expone una operación de borrado; un documento generado nunca se elimina. El paciente solo puede marcar como oculto_paciente un documento externo que él mismo adjuntó — el registro permanece en la base de datos.

NOTA

El uso de NOT VALID en los constraints de rango permite desplegar la migración sin fallar sobre datos legados que aún no cumplan la regla. Para exigir la validación también sobre el histórico, debe ejecutarse VALIDATE CONSTRAINT de forma separada tras sanear esos datos.

5. Requisitos del Sistema

5.1 Entorno de Desarrollo Local

Componente	Especificación
Node.js	Backend: sin cota estricta declarada (recomendado ≥ 18 LTS). Frontend: $\geq 20.0.0$ (declarado en client/package.json $\rightarrow$ engines.node).
npm	Incluido con Node.js. Se usan package-lock.json en ambas carpetas: usar npm ci en entornos reproducibles.
PostgreSQL	$\geq 14.x$. Instancia local o remota accesible con las credenciales configuradas en .env.
Git	$\geq 2.x$ para clonar el repositorio y gestionar ramas.
Editor	Visual Studio Code (recomendado), con ESLint habilitado (client/eslint.config.js).
Navegador	Chrome, Edge o Firefox en versión reciente, para pruebas de la SPA y del visor PDFSlick.
Sistema operativo	Windows 10/11, macOS 12+ o Ubuntu 20.04+.

5.2 Entorno de Producción (Railway)

Componente	Descripción
Plataforma	Railway PaaS — https://railway.app
Servicios	Dos servicios independientes dentro del mismo proyecto Railway: uno para client/ y otro para server/, cada uno con su propio Root Directory.
Builder (cliente)	NIXPACKS, definido en client/railway.json, con buildCommand npm run build y startCommand npm start (sirve dist/ con el paquete serve).
Builder (servidor)	NIXPACKS por detección automática de Railway; startCommand node src/server.js (script start de server/package.json).
Base de datos	Plugin PostgreSQL administrado por Railway, provisto solo en el servicio server.
Red	HTTPS con certificado TLS provisto automáticamente por Railway en ambos servicios.

6. Instalación y Configuración del Entorno Local

6.1 Clonar el Repositorio

```
git clone https://github.com/Imjuanisss/proyecto-melika.git
cd proyecto-melika
```

6.2 Backend — server/

Paso 1 — Instalar dependencias

```
cd server
npm install
```

Paso 2 — Configurar variables de entorno

Crear un archivo .env dentro de server/ con las siguientes variables. Ninguna de ellas está versionada en el repositorio (correctamente excluidas vía .gitignore):

Variable	Descripción
DB_HOST	Host del servidor PostgreSQL.
DB_PORT	Puerto de PostgreSQL (ej. 5432).
DB_USER	Usuario de la base de datos.
DB_PASSWORD	Contraseña del usuario de la base de datos.
DB_NAME	Nombre de la base de datos (ej. melika_db).
JWT_SECRET	Clave secreta para firmar y verificar los tokens JWT. Debe ser un valor largo y aleatorio.
FRONTEND_URL	URL pública del frontend, usada por cors() para validar el origen de las peticiones.
PORT	Puerto en el que escucha Express (ej. 3000). En Railway lo asigna la plataforma automáticamente.
GOOGLE_CLIENT_ID	Client ID de OAuth2 de la aplicación registrada en Google Cloud Console.
GOOGLE_CLIENT_SECRET	Client Secret de la misma aplicación OAuth2.
GOOGLE_REFRESH_TOKEN	Refresh token de larga duración generado una única vez con la cuenta de Gmail remitente.
EMAIL_USER	Cuenta de Gmail que envía las notificaciones.

Variable	Descripción
EMAIL_FROM	Opcional. Nombre visible del remitente; por defecto "MELIKA Salud <EMAIL_USER>".

ADVERTENCIA

El envío de correo en MELIKA NO usa SMTP (nodemailer con service:'gmail' o smtp.gmail.com). Usa la Gmail API sobre HTTPS mediante OAuth2, porque Railway bloquea a nivel de firewall los puertos SMTP salientes (25, 465, 587). Ver sección 7.5 para el detalle completo de esta decisión de arquitectura.

Paso 3 — Crear la base de datos y cargar el esquema

```
psql -U postgres -c "CREATE DATABASE melika_db;"
psql -U postgres -d melika_db -f server/src/database/melika_db.sql
```

El archivo melika_db.sql contiene el esquema base y todas las migraciones incrementales aplicadas hasta la fecha (columnas de signos vitales, documentos clínicos, constraints de integridad, etc.), por lo que ejecutarlo completo sobre una base nueva deja el esquema en su estado más reciente.

Paso 4 — Crear el primer usuario administrador

```
node scripts/crearAdmin.js
```

El script verifica si ya existe un administrador con el correo configurado antes de crear uno nuevo, por lo que es seguro ejecutarlo más de una vez.

CRÍTICO

El script crearAdmin.js define una contraseña temporal en texto plano dentro del propio código fuente. Es indispensable cambiarla inmediatamente después del primer inicio de sesión, y considerar mover ese valor a una variable de entorno antes de reutilizar el script en un entorno compartido.

Paso 5 — Iniciar el servidor de desarrollo

```
npm run dev
```

El script dev ejecuta nodemon --watch src/server.js. La respuesta esperada en http://localhost:3000/ es un JSON con status: "ok" y el mensaje "Servidor MELIKA funcionando correctamente".

6.3 Frontend — client/**Paso 1 — Instalar dependencias**

```
cd client
npm install
```

Paso 2 — Configurar variables de entorno

Crear un archivo .env en client/ con:

Variable	Descripción
VITE_API_URL	URL base del backend (ej. http://localhost:3000 en desarrollo, o el dominio de Railway del servicio server en producción). Si no se define, apiClient.js usa http://localhost:3000 por defecto.

Paso 3 — Iniciar el servidor de desarrollo

```
npm run dev
```

Vite expondrá la aplicación en http://localhost:5173. Este es, precisamente, uno de los orígenes que el backend acepta por defecto en su configuración de CORS (junto con http://127.0.0.1:5173 y FRONTEND_URL).

7. Despliegue en Railway

MELIKA se despliega íntegramente en Railway mediante dos servicios independientes dentro de un mismo proyecto, más el plugin de PostgreSQL. Esta separación permite escalar, reiniciar y versionar el frontend y el backend de forma completamente independiente.

7.1 Servicio del Backend (server)

1. Crear un proyecto en <https://railway.app> y añadir un servicio desde el repositorio de GitHub.
2. En la configuración del servicio, establecer server como Root Directory.
3. Añadir el plugin de PostgreSQL al proyecto; Railway inyecta automáticamente variables de conexión propias (que pueden referenciarse o mapearse a DB_HOST, DB_PORT, DB_USER, DB_PASSWORD, DB_NAME).
4. Configurar en el servicio las variables restantes: JWT_SECRET, FRONTEND_URL, GMAIL_CLIENT_ID, GMAIL_CLIENT_SECRET, GMAIL_REFRESH_TOKEN, EMAIL_USER, EMAIL_FROM.
5. Railway detecta el script start (node src/server.js) y lo ejecuta automáticamente al finalizar el build.
6. Ejecutar el esquema/migraciones sobre la base de datos de producción una vez el servicio esté activo, usando la Railway Shell o una conexión directa con psql.
7. Ejecutar node scripts/crearAdmin.js una única vez para crear el primer administrador.
8. Railway asigna automáticamente un dominio público con certificado TLS para este servicio.

7.2 Servicio del Frontend (client)

El frontend se despliega como un segundo servicio Railway, configurado explícitamente mediante client/railway.json:

```
{
  "$schema": "https://railway.app/railway.schema.json",
  "build": {
    "builder": "NIXPACKS",
    "buildCommand": "npm run build"
  },
  "deploy": {
    "startCommand": "npm start",
    "restartPolicyType": "ON_FAILURE"
  }
}
```

El comando npm start no levanta Vite en modo desarrollo: ejecuta serve -s dist -l \$PORT, sirviendo el build estático de producción (dist/) generado por vite build, con enrutamiento en modo SPA (flag -s) para que las rutas de react-router-dom funcionen correctamente al refrescar el navegador.

9. Añadir un segundo servicio en el mismo proyecto Railway, apuntando al mismo repositorio con client como Root Directory.
10. Configurar la variable VITE_API_URL con el dominio público que Railway asignó al servicio server.
11. Railway ejecuta npm run build durante el build y luego npm start para servir la aplicación.
12. Una vez desplegado, actualizar FRONTEND_URL en el servicio server con el dominio público de este servicio, para que CORS lo acepte.

ADVERTENCIA

Las variables VITE_API_URL (frontend) y FRONTEND_URL (backend) deben apuntar una a la otra de forma cruzada y exacta, incluyendo el esquema https://. Un desajuste entre ambas es la causa más común de errores de CORS en producción (ver sección 12).

7.3 Orden Recomendado de Despliegue Inicial

13. Desplegar primero el servicio server con el plugin de PostgreSQL.
14. Ejecutar el esquema SQL y crearAdmin.js sobre la base de datos ya activa.
15. Anotar el dominio público generado para server.
16. Desplegar el servicio client con VITE_API_URL apuntando a ese dominio.
17. Volver al servicio server y fijar FRONTEND_URL con el dominio de client.
18. Redesplegar server para que la nueva variable de entorno surta efecto en la validación de CORS.

7.4 Envío de Correo: Gmail API en lugar de SMTP

Esta es una de las decisiones de arquitectura más importantes del backend y está documentada directamente en el código fuente de emailService.js. Railway, como la mayoría de plataformas cloud modernas, bloquea a nivel de firewall los puertos SMTP salientes (25, 465, 587) para mitigar el uso de su infraestructura en el envío masivo de spam.

Como consecuencia, cualquier intento de usar Nodemailer con service: 'gmail' o smtp.gmail.com está condenado a fallar en producción sin importar cuántos parámetros de red se ajusten (IPv4 forzado, timeouts, puertos alternativos). La solución adoptada en MELIKA es enviar el correo a través de la Gmail API, que viaja sobre HTTPS (puerto 443) — el mismo puerto que cualquier llamada REST normal — por lo que nunca es bloqueada.

Flujo de autenticación OAuth2 utilizado

- Se crea un cliente OAuth2 (google.auth.OAuth2) con GMAIL_CLIENT_ID y GMAIL_CLIENT_SECRET.
- Se le inyecta un GMAIL_REFRESH_TOKEN de larga duración, generado una única vez fuera del flujo normal de la aplicación (por ejemplo, con OAuth 2.0 Playground).

- El mensaje se construye en formato RFC 2822, con el asunto codificado en Base64 (RFC 2047) para soportar tildes y la letra ñ sin corromperse, y se codifica en base64url en el campo raw que exige la Gmail API.
- Se invoca `gmail.users.messages.send()` para el envío final.

NOTA

Si el envío de correo empieza a fallar en producción, el diagnóstico casi siempre apunta a `GMAIL_REFRESH_TOKEN` vencido o revocado, o a que el proyecto de Google Cloud asociado perdió el scope de Gmail. Ver sección 12.

8. Autenticación y Seguridad

8.1 Flujo de Autenticación

El registro de un paciente sigue un flujo de verificación por correo antes de habilitar la cuenta. Los médicos y administradores, en cambio, se incorporan mediante un token de invitación generado por un administrador existente (tabla `tokens_invitacion`), no por autorregistro.

19. Registro (POST `/auth/register`): el usuario envía sus datos; la contraseña se almacena únicamente como hash (bcrypt); la cuenta queda inactiva y no verificada.
20. Verificación (GET `/auth/verify-code`): se genera un código de 6 dígitos con expiración corta, almacenado en `codigos_verificacion`. Al validarlo, la cuenta pasa a `verificado = true`.
21. Reenvío de código (POST `/auth/reenviar-codigo`): disponible si el código expiró o no llegó.
22. Login (POST `/auth/login`): valida credenciales contra el hash almacenado y emite un JWT firmado con `JWT_SECRET`, que incluye el id de usuario y su rol.
23. Recuperación de contraseña (POST `/auth/recuperar-password` → POST `/auth/nueva-password`): mismo mecanismo de código temporal, aplicado al flujo de restablecimiento.

8.2 Autorización por Rol

La autorización se implementa en `server/src/middleware/authMiddleware.js` mediante cuatro funciones que se componen explícitamente en cada archivo de rutas:

Middleware	Función
<code>verifyToken</code>	Exige el header <code>Authorization: Bearer <token></code> , verifica la firma y expiración del JWT con <code>JWT_SECRET</code> y adjunta el payload decodificado a <code>req.usuario</code> . Responde 401 si el token falta, es inválido o expiró.
<code>isAdmin</code>	Exige que <code>req.usuario.rol</code> sea <code>'admin'</code> . Responde 403 en caso contrario.
<code>isMedico</code>	Exige que <code>req.usuario.rol</code> sea <code>'medico'</code> . Responde 403 en caso contrario.
<code>isPaciente</code>	Exige que <code>req.usuario.rol</code> sea <code>'paciente'</code> . Responde 403 en caso contrario.

Este diseño hace que la política de acceso de cada endpoint sea legible directamente en su definición de ruta, por ejemplo:

```
router.get('/agenda', verifyToken, isMedico, agendaMedico);
```

En `adminRoutes.js`, en lugar de repetir el middleware en cada línea, se aplica una única vez a nivel de router con `router.use(verifyToken, isAdmin)`, dado que todas las rutas de ese archivo son exclusivas de administrador.

8.3 Buenas Prácticas de Seguridad Implementadas

- Contraseñas con hashing `bcrypt` (factor de costo 12 en `crearAdmin.js`), nunca almacenadas ni registradas en texto plano en la base de datos.
- Tokens JWT con expiración; el cliente debe volver a autenticar al usuario cuando expiran (ver `troubleshooting`, sección 12).
- CORS restringido por lista blanca de orígenes (`localhost:5173`, `127.0.0.1:5173` y `FRONTEND_URL`), no un comodín (*), con `credentials: true` habilitado solo para esos orígenes.
- Tokens de invitación de un solo uso (columna `usado`) y con expiración, para la incorporación de médicos y administradores.
- Auditoría de citas: la tabla `logs_citas` conserva un snapshot JSONB del estado anterior y posterior de cada cambio, útil para trazabilidad ante disputas o incidentes.

CRÍTICO

La contraseña temporal del script `crearAdmin.js` está escrita en texto plano en el código fuente del repositorio. Se recomienda externalizarla a una variable de entorno (p. ej. `ADMIN_TEMP_PASSWORD`) antes de ejecutar el script en cualquier entorno con acceso compartido al repositorio, y rotarla inmediatamente tras el primer uso.

9. Documentación de la API REST

9.1 Convenciones Generales

- URL base: la definida por VITE_API_URL en el cliente (p. ej. http://localhost:3000 en desarrollo).
- Formato: JSON en request y response (express.json()); no se usa multipart en los endpoints documentados aquí.
- Autenticación: header Authorization: Bearer <token_jwt> en todo endpoint marcado como protegido.
- Errores: el cliente (apiClient.js) normaliza toda respuesta no exitosa a un objeto con message, mensaje, errores (array de validaciones específicas cuando aplica, p. ej. en historias clínicas) y status (código HTTP).
- Orden de rutas: en todos los routers del proyecto, los segmentos estáticos (/admin, /categorias, /mis-citas) se declaran siempre antes que los segmentos dinámicos (/:id). Ver sección 11.1 para el detalle de por qué esta regla es crítica en Express.

9.2 Módulo de Autenticación — /auth

Método	Ruta	Auth	Descripción
POST	/auth/register	No	Registra un nuevo paciente. Password se almacena con hash bcrypt.
POST	/auth/login	No	Valida credenciales y emite el JWT.
GET	/auth/verify-code	No	Verifica el código de 6 dígitos enviado por correo tras el registro.
POST	/auth/reenviar-codigo	No	Reenvía un nuevo código de verificación.
POST	/auth/recuperar-password	No	Inicia el flujo de recuperación de contraseña por correo.
POST	/auth/nueva-password	No	Aplica la nueva contraseña usando el token/código de recuperación.

9.3 Módulo de Administración — /admin

Todas las rutas de este módulo exigen verifyToken + isAdmin, aplicado una sola vez a nivel de router.

Método	Ruta	Descripción
GET	/admin/stats	Métricas agregadas del negocio para el dashboard administrativo.
GET	/admin/usuarios	Lista todos los usuarios registrados en la plataforma.
PATCH	/admin/usuarios/:id/estado	Activa o desactiva la cuenta de un usuario.
GET	/admin/citas	Lista todas las citas de todos los pacientes.
PATCH	/admin/citas/:id/estado	Cambia el estado de una cita desde el panel admin.
GET	/admin/horarios	Lista las franjas horarias configuradas.
POST	/admin/horarios/masivo	Creación en bloque de franjas para una semana completa. Declarada antes de /horarios/:id por la regla de orden de rutas.
POST	/admin/horarios	Creación de una franja individual (mantenida por compatibilidad).
PUT	/admin/horarios/:id	Edita una franja horaria existente.
DELETE	/admin/horarios/:id	Elimina una franja horaria.
POST	/admin/especialidades	Crea una nueva especialidad médica.
PUT	/admin/especialidades/:id	Actualiza una especialidad existente.
PATCH	/admin/especialidades/:id/estado	Activa o desactiva una especialidad del catálogo.
POST	/admin/medicamentos	Crea un nuevo medicamento en el catálogo.
PUT	/admin/medicamentos/:id	Actualiza un medicamento existente.
PATCH	/admin/medicamentos/:id/estado	Activa o desactiva un medicamento.

9.4 Módulo de Especialidades — /especialidades

Método	Ruta	Auth	Descripción
GET	/especialidades	No	Lista especialidades activas para el catálogo público.
GET	/especialidades/admin	No*	Lista todas las especialidades, incluidas las inactivas, para el panel admin.
GET	/especialidades/disponibilidad	No	Consulta disponibilidad de franjas para una especialidad en una fecha.
GET	/especialidades/disponibilidad-rango	No	Igual a la anterior, para un rango de fechas (vista de calendario).
POST	/especialidades	No*	Crea una especialidad.
PUT	/especialidades/:id	No*	Actualiza una especialidad.

Método	Ruta	Auth	Descripción
GET	/especialidades/:id/medicos	No	Lista los médicos asociados a una especialidad.

ADVERTENCIA

Las rutas de escritura de /especialidades (POST, PUT) no declaran verifyToken/isAdmin directamente en especialidadesRoutes.js, a diferencia del resto del sistema. Se recomienda auditar este router y alinearlos con el patrón de protección explícita usado en adminRoutes.js y medicosRoutes.js, para evitar exponer operaciones de escritura del catálogo sin autenticación.

9.5 Módulo de Médicos — /medicos y /medico

Ambos prefijos apuntan al mismo router (medicosRoutes.js); las rutas internas distinguen el contexto de uso: /medicos para operaciones administrativas sobre el recurso médico, /medico para el propio médico autenticado gestionando su perfil, agenda y franjas.

Método	Ruta	Auth	Descripción
POST	/medicos/activar	No	El médico activa su cuenta con el token de invitación enviado por el admin.
GET	/medicos	Admin	Lista todos los médicos.
POST	/medicos	Admin	Crea un nuevo médico (usuario + perfil profesional).
PUT	/medicos/:id	Admin	Actualiza los datos de un médico.
PATCH	/medicos/:id/estado	Admin	Activa o desactiva un médico.
GET	/medico/perfil	Médico	Perfil profesional del médico autenticado.
GET	/medico/agenda/rango	Médico	Agenda del médico para un rango de fechas (consumida por FullCalendar).

Método	Ruta	Auth	Descripción
GET	/medico/agenda	Médico	Agenda diaria del médico.
PATCH	/medico/citas/:id/gestionar	Médico	Cierra el resultado clínico de una cita: estado completada o no_asistio + notas_medicas. Implementada en historiasController, reexpuesta aquí.
GET	/medico/franjas	Médico	Lista las franjas de disponibilidad propias.
POST	/medico/franjas	Médico	Crea una franja de disponibilidad propia.
PATCH	/medico/franjas/:id	Médico	Edita una franja propia.
DELETE	/medico/franjas/:id	Médico	Elimina una franja propia.

9.6 Módulo de Citas — /citas

Método	Ruta	Auth	Descripción
GET	/citas/mis-citas	Sí	Citas del usuario autenticado (paciente o médico, según rol).
GET	/citas/calendario	Sí	Vista de citas en formato calendario.
POST	/citas	Sí	Crea una nueva cita, validando disponibilidad de la franja horaria.
PATCH	/citas/:id	Sí	Cancela una cita (transición de estado, no elimina el registro).
DELETE	/citas/:id	Sí	Elimina una cita.

Ciclo de vida del estado de una cita

```

pendiente → completada
           ↳ no_asistio
pendiente → cancelada   (en cualquier momento antes de la atención)

```

9.7 Módulo de Historias Clínicas y Documentos — /historias

Este es el módulo con las reglas de orden de rutas más estrictas del proyecto, documentadas directamente como comentario en el propio archivo de rutas: los segmentos estáticos (/paciente/:id, /documentos/...) deben declararse siempre antes que la ruta dinámica genérica /:id, porque de lo contrario Express la capturaría primero e impediría alcanzar las rutas estáticas.

Método	Ruta	Auth	Descripción
GET	/historias/paciente/:id_paciente	Sí	Historial clínico completo de un paciente.

Método	Ruta	Auth	Descripción
GET	/historias/cita/:id_cita	Sí	Historia clínica asociada a una cita específica.
POST	/historias	Médico	Crea la historia_principal de una cita.
GET	/historias/:id/completa	Sí	Historia clínica completa por su id, incluyendo recetas y órdenes.
POST	/historias/:id/aclaracion	Médico	Registra una nota de aclaración/evolución sobre una historia existente.
PUT	/historias/:id	Médico	Alias del endpoint anterior, mantenido por compatibilidad con consumidores existentes.
PATCH	/historias/gestionar-cita/:id	Médico	Cierra el resultado clínico de una cita (mismo controlador que /medico/citas/:id/gestionar).
GET	/historias/documentos/paciente/:id_paciente	Sí	Lista los documentos clínicos de un paciente (historias, fórmulas, órdenes, documentos externos).
GET	/historias/documentos/:id	Sí	Detalle de un documento clínico.
POST	/historias/documentos	Sí	Registra un nuevo documento clínico.
PATCH	/historias/documentos/:id/ocultar	Paciente	El paciente oculta (no elimina) un documento externo que él mismo adjuntó.

CRÍTICO

La ruta POST
/historias/:id/aclaracion
fue añadida para
corregir un defecto real
detectado en
producción: el frontend
(FormularioAclaracion.js
x y
ModalHistoriaClinica.js
x) invocaba ese endpoint

Método	Ruta	Auth	Descripción
desde el inicio, pero el backend únicamente exponía PUT /historias/:id. El resultado era un 404 silencioso cada vez que un médico intentaba registrar una nota de aclaración o evolución. Ambas rutas están intencionalmente mapeadas al mismo controlador para no romper compatibilidad.			

9.8 Módulo de Medicamentos — /medicamentos

Método	Ruta	Descripción
GET	/medicamentos/admin	Lista completa para el panel admin (incluye inactivos). Declarada antes de /:id por la regla de orden de rutas.
GET	/medicamentos/categorias	Lista las categorías disponibles para filtros del catálogo.
GET	/medicamentos	Catálogo público de medicamentos activos, con soporte de búsqueda.
GET	/medicamentos/:id	Ficha técnica de un medicamento.
POST	/medicamentos	Crea un nuevo medicamento (uso administrativo).
PUT	/medicamentos/:id	Actualiza un medicamento existente.

10. Generación de Documentos Clínicos en PDF

La generación de historias clínicas, fórmulas médicas y órdenes de examen en PDF es una de las funcionalidades más elaboradas de MELIKA, tanto para la vista de paciente como para la de médico. Se apoya en dos librerías con responsabilidades distintas y complementarias.

10.1 Librerías Utilizadas

Librería	Responsabilidad
@react-pdf/renderer	Motor de generación: construye el PDF de forma vectorial a partir de componentes React (Document, Page, Text, View, StyleSheet) — no renderiza HTML/CSS a PDF, sino que dibuja el documento con su propio motor de layout.
@pdfslick/react	Motor de visualización: renderiza el PDF ya generado dentro de la propia aplicación (basado en PDF.js), sin obligar al usuario a descargar el archivo para consultarlo.

10.2 Arquitectura: Generación en el Cliente

A diferencia de un enfoque tradicional donde el backend genera el PDF y lo entrega como archivo, en MELIKA el documento se construye enteramente en el navegador, a partir de los datos clínicos que la API ya autorizó y devolvió al cliente. El flujo implementado en HistorialPaciente.jsx es:

24. El componente obtiene los datos de la historia clínica desde la API (ya filtrados por los permisos del usuario autenticado).
25. Esos datos se pasan como props a una plantilla (PlantillaHistoriaPDF, PlantillaFormulaPDF o PlantillaExamenPDF), construida con los primitivos de @react-pdf/renderer.
26. Se invoca pdf(documento).toBlob() para generar el binario del PDF en memoria, en el propio navegador.
27. Se crea una URL temporal con URL.createObjectURL(blob), que apunta a ese binario sin necesidad de subirlo a ningún servidor.
28. Esa URL se entrega a VisorPDFModal, que la renderiza in-app mediante @pdfslick/react.

NOTA

Generar el PDF en el cliente evita duplicar la lógica de maquetación clínica en el backend y reduce la carga del servidor, ya que cada descarga o visualización es un cómputo local del navegador del usuario que ya tiene los datos autorizados. El backend solo necesita servir los datos estructurados; nunca genera ni almacena el binario del PDF.

CRÍTICO

Es fundamental no confundir este flujo con generar el blob a partir de una URL remota o de datos no verificados: el blob se construye siempre localmente, a partir de la respuesta ya autenticada y autorizada de la API. Tratar la URL del PDF como si fuera un recurso persistente en el backend (por ejemplo, intentando reutilizarla tras un recambio de página) es un error de arquitectura, ya que `URL.createObjectURL` crea una referencia válida solo mientras dura la sesión del documento en memoria.

10.3 Configuración Especial de Vite

`@pdfslick/react` ejecuta `PDF.js` en un Web Worker, y esto exige ajustes específicos en `vite.config.js` que no son evidentes a simple vista y que, de omitirse, provocan errores de resolución de módulos en tiempo de build:

```
worker: {
  format: 'es',    // El worker de PDF.js requiere formato ESM
},
resolve: {
  alias: {
    // Redirige la ruta defectuosa del shim de use-sync-external-store
    // hacia un puente ESM propio del proyecto
    'use-sync-external-store/shim/with-selector.js':
      path.resolve('./src/shim-with-selector.js'),
    'use-sync-external-store/shim': 'react',
  },
},
optimizeDeps: {
  include: [
    '@react-pdf/renderer',
    'use-sync-external-store/with-selector.js',
  ],
  exclude: ['@pdfslick/react'], // protege el worker nativo de @pdfslick
},
```

ADVERTENCIA

Si al actualizar dependencias reaparecen errores de resolución relacionados con `use-sync-external-store` o con el worker de `PDF.js`, revisar primero esta sección de `vite.config.js` antes de investigar otras causas: son ajustes deliberados, no configuración por defecto de Vite.

10.4 Cumplimiento Normativo del Documento Generado

Las plantillas de historia clínica (`PlantillaHistoriaPDF.jsx`) están diseñadas para cumplir la normativa colombiana de historia clínica — Resolución 1995 de 1999 y Ley 2015 de 2020 —, lo que se refleja directamente en el modelo de datos: el cierre legal obligatorio (nombre y número RETHUS del médico, ver sección 4.3) y la inmutabilidad de los documentos clínicos generados no son solo reglas de negocio, sino requisitos de trazabilidad legal que el PDF final materializa.

11. Convenciones de Ingeniería del Proyecto

11.1 Orden de Rutas en Express: Estático Antes de Dinámico

Es la convención más repetida y mejor documentada del backend, presente como comentario explícito en `historiasRoutes.js`, `medicosRoutes.js`, `medicamentosRoutes.js`, `especialidadesRoutes.js` y `adminRoutes.js`. Express resuelve las rutas en el orden en que se declaran; si una ruta dinámica como `/:id` se declara antes que una ruta estática como `/categorias`, Express interpretará "categorias" como si fuera un valor de `:id`, y la ruta estática nunca será alcanzada.

Regla aplicada de forma consistente en todo el proyecto:

```
// Correcto
router.get('/categorias', listarCategorias);
router.get('/:id', obtenerMedicamento);

// Incorrecto - /categorias jamás se alcanza
router.get('/:id', obtenerMedicamento);
router.get('/categorias', listarCategorias);
```

Todo nuevo endpoint que añada un segmento fijo a un router existente debe declararse antes de cualquier segmento dinámico de ese mismo router.

11.2 CSS Puro y Arquitectura de Tokens

Por decisión de proyecto, MELIKA no incorpora frameworks de UI (Tailwind, Bootstrap, ni librerías de componentes). Todo el estilado se construye con CSS puro, organizado bajo una convención de nomenclatura tipo BEM con prefijos por dominio — por ejemplo, las clases `adm-*` del panel de administración —, y tokens de diseño centralizados en `client/src/styles/index.css`.

- Cada página o componente complejo tiene su propio archivo `.css` homónimo (`Catalogo.jsx ↔ Catalogo.css`, `AdminLayout.jsx ↔ AdminLayout.css`).
- Los módulos del panel admin comparten una hoja base (`admin-shared.css`) además de sus estilos particulares, para mantener coherencia visual entre gestión de médicos, especialidades, medicamentos, citas, horarios y usuarios.
- Al integrar código de distintos colaboradores en un mismo panel, es indispensable verificar que las rutas relativas de import de CSS sean correctas y que no existan colisiones de nombres de clase entre módulos independientes.

11.3 Inmutabilidad de Historias Clínicas

Como se detalla en la sección 4.3, ninguna historia clínica principal se edita ni se sobrescribe una vez creada. Toda corrección posterior se registra como un nuevo documento (nota_aclaracion o nota_evolucion) vinculado a la misma cita, preservando el registro original íntegro. Este principio de append-only se extiende también a la tabla documentos_clinicos: no existe operación de DELETE sobre documentos clínicos generados, solo la posibilidad de que el paciente oculte de su propia vista un documento externo que él mismo adjuntó.

12. Ejecución de Pruebas

Al cierre de esta versión del manual, el proyecto no cuenta con una suite de pruebas automatizadas configurada en package.json (no existen scripts test en client/ ni en server/). La validación actual del sistema es manual, apoyada en ESLint para el frontend.

12.1 Verificación Estática (Frontend)

```
cd client
npm run lint
```

Ejecuta ESLint sobre todo el proyecto con la configuración de client/eslint.config.js, incluyendo las reglas de eslint-plugin-react-hooks y eslint-plugin-react-refresh.

12.2 Recomendación de Cobertura de Pruebas

Dada la sensibilidad clínico-legal del sistema (inmutabilidad de historias, constraints de integridad, control de acceso por rol), se recomienda priorizar la incorporación progresiva de:

- Pruebas unitarias sobre los controladores del módulo de historias clínicas (historiasController.js), en particular las funciones de validación en validacionesHistoria.js.
- Pruebas de integración sobre el middleware de autenticación y autorización (authMiddleware.js), verificando los tres roles (paciente, médico, admin) y sus rechazos correspondientes (401/403).
- Pruebas end-to-end del flujo completo de agendamiento → atención → cierre de cita → generación de historia clínica → generación de PDF.

NOTA

Al incorporar una suite de pruebas, seguir el mismo patrón documentado en el ecosistema Node/Express del proyecto: usar pool.end() en un hook de cierre para liberar las conexiones de PostgreSQL y evitar que el proceso de pruebas quede colgado (--detectOpenHandles), y ejecutar las pruebas de base de datos en serie para evitar condiciones de carrera sobre el mismo esquema.

13. Procedimientos de Mantenimiento

Tarea	Procedimiento
Actualización de dependencias	Ejecutar npm audit en client/ y server/ periódicamente. Actualizar con npm update, verificar client/package.json → engines.node al subir de versión de Node, y validar manualmente los flujos críticos (login, agendamiento, generación de PDF) tras cada actualización.
Backup de base de datos	Utilizar el panel de backup del plugin de PostgreSQL en Railway. Dada la naturaleza clínico-legal de los datos, se recomienda backup diario y retención extendida.
Rotación de secretos	Rotar JWT_SECRET periódicamente — invalida todos los tokens activos, por lo que debe comunicarse a los usuarios. Rotar también GMAIL_REFRESH_TOKEN si se sospecha compromiso de la cuenta de correo.
Nuevas migraciones	Añadir el nuevo bloque SQL al final de server/src/database/melika_db.sql (o a un archivo de migración separado si el equipo adopta ese patrón), siempre con IF NOT EXISTS / ADD COLUMN IF NOT EXISTS / NOT VALID donde aplique, para mantener la reejecución segura sobre entornos existentes.
Gestión de imágenes estáticas	Las imágenes de especialidades, médicos y medicamentos residen en client/public/imagenes/. Al añadir nuevas, mantener la convención de subcarpetas por dominio (especialidades/, medicamentos/, medicos/) y evitar nombres de archivo ambiguos.
Monitoreo en Railway	Revisar los logs de ambos servicios (client y server) ante respuestas 5xx. Configurar Health Checks apuntando a GET / del servicio server, que responde un JSON de estado.
Auditoría de citas	La tabla logs_citas crece de forma append-only con cada cambio de estado. Establecer una política de retención o archivado periódico si el volumen de citas lo justifica.

14. Resolución de Problemas Frecuentes

Problema	Solución
Error de conexión a la base de datos	Verificar DB_HOST, DB_PORT, DB_USER, DB_PASSWORD y DB_NAME en .env o en las variables del servicio server en Railway. Confirmar que el plugin de PostgreSQL esté activo y que el servicio server tenga permitido conectarse a él dentro del mismo proyecto Railway.
Token JWT inválido o expirado (401)	El usuario debe volver a iniciar sesión. Verificar que JWT_SECRET sea idéntico entre el entorno donde se emitió el token y el entorno donde se valida (un JWT_SECRET distinto en local y en Railway invalida todos los tokens emitidos en el otro entorno).
Acceso denegado con rol correcto (403)	Revisar el payload del JWT decodificado: el campo rol debe coincidir exactamente con 'admin', 'medico' o 'paciente' (sensible a mayúsculas/minúsculas y espacios). Confirmar también que el middleware de rol esté aplicado en el orden correcto (verifyToken siempre antes que isAdmin/isMedico/isPaciente).
Ruta estática nunca se alcanza (404 inesperado)	Síntoma típico: un endpoint como /categorias o /admin devuelve 404 o es interpretado como un :id. Revisar el orden de declaración en el archivo de rutas correspondiente — la ruta estática debe declararse antes que la ruta dinámica (ver sección 11.1).
Correos no enviados	Verificar GMAIL_CLIENT_ID, GMAIL_CLIENT_SECRET, GMAIL_REFRESH_TOKEN y EMAIL_USER. Un refresh token vencido o revocado en Google Cloud Console es la causa más frecuente. Confirmar también que el proyecto de Google Cloud conserve el scope de Gmail habilitado para esa cuenta OAuth2.
CORS bloqueado en el frontend	Confirmar que FRONTEND_URL en el servicio server coincida exactamente (incluido https://, sin barra final) con el dominio público del servicio client. En desarrollo local, confirmar que el frontend corre efectivamente en localhost:5173 o 127.0.0.1:5173.
Errores de build relacionados con use-sync-external-store o el worker de PDF.js	Revisar la sección de resolve.alias, worker y optimizeDeps de client/vite.config.js (ver sección 10.3): son ajustes deliberados para @pdfslick/react, no configuración por defecto de Vite.
El PDF generado no se ve o el visor queda en blanco	Confirmar que pdf(documento).toBlob() se resolvió correctamente antes de pasar la URL a VisorPDFModal, y que la URL creada con URL.createObjectURL no fue revocada (URL.revokeObjectURL) antes de que el visor terminara de cargarla.
No se puede crear una historia clínica principal	El backend rechazará el guardado si falta algún campo obligatorio exigido por el constraint chk_historia_principal_completa (anamnesis, antecedentes patológicos, antecedentes alérgicos, examen físico, diagnóstico CIE-10 y su descripción, plan de tratamiento, firma del médico). Revisar el array errores que devuelve la API en la respuesta 422.
Falla la creación de una segunda historia sobre la misma cita	Comportamiento esperado si se intenta crear otra historia_principal para una cita que ya tiene una (protegido por el índice único parcial

Problema	Solución
	uq_historia_principal_por_cita). Para registrar una corrección, usar el endpoint de aclaración/evolución en lugar de crear una historia principal nueva.
El script crearAdmin.js indica que el admin ya existe	Comportamiento esperado: el script es idempotente y no crea un segundo administrador con el mismo correo. Si se necesita restablecer la contraseña, hacerlo desde el flujo de recuperación de contraseña o directamente en base de datos.

15. Control de Versiones

El historial de cambios del proyecto se documenta en CHANGELOG.md, en la raíz del repositorio, siguiendo el formato de Keep a Changelog y Versionado Semántico (MAJOR.MINOR.PATCH).

15.1 Versión Actual

Versión	Fecha	Resumen
1.0.0	2026-07-03	Primera versión estable de MELIKA: gestión de historias clínicas, ciclo completo de citas médicas, módulo de aclaraciones clínicas, generación y visualización de documentos en PDF, autenticación con control de acceso por rol, backend con connection pooling sobre PostgreSQL, y despliegue unificado en Railway.

15.2 Correcciones Incorporadas Antes del Lanzamiento

La auditoría integral previa al release 1.0.0 identificó y corrigió, entre otros, los siguientes defectos, útiles como referencia si reaparecen síntomas similares tras cambios futuros:

- Ruta rota en el flujo de cancelación de citas, que impedía completar la operación desde el cliente.
- FormularioAclaracion.jsx existía en el código pero nunca se montaba en el árbol de React, dejando la funcionalidad inaccesible pese a estar completamente implementada.
- El evento personalizado melika:aclaracion-creada se emitía correctamente pero no tenía listener, por lo que la interfaz no se refrescaba tras registrar una aclaración.

15.3 Convención para Nuevas Entradas

Toda nueva versión debe añadirse en CHANGELOG.md bajo las categorías estándar (Added, Changed, Fixed, Removed), con la fecha en formato ISO (AAAA-MM-DD) y una descripción orientada a impacto funcional, no solo a la lista de commits.

16. Anexos

16.1 Glosario de Estados de una Cita

Estado	Significado
pendiente	Cita creada, aún no atendida.
completada	El médico cerró la cita registrando el resultado clínico.
cancelada	La cita fue cancelada antes de la atención.
no_asistio	El paciente no se presentó a la cita.

16.2 Glosario de Tipos de Registro Clínico

Tipo	Significado
historia_principal	Registro clínico original de la consulta. Único por cita.
nota_aclaracion	Corrección o precisión posterior sobre una historia ya creada, sin alterar el original.
nota_evolucion	Registro de evolución del paciente en atenciones o seguimientos posteriores.

16.3 Consolidado de Variables de Entorno

server/.env

```
DB_HOST=  
DB_PORT=  
DB_USER=  
DB_PASSWORD=  
DB_NAME=  
JWT_SECRET=  
FRONTEND_URL=  
PORT=  
GMAIL_CLIENT_ID=  
GMAIL_CLIENT_SECRET=  
GMAIL_REFRESH_TOKEN=  
EMAIL_USER=  
EMAIL_FROM=
```

client/.env

```
VITE_API_URL=
```

16.4 Referencias

- Repositorio del proyecto: github.com/Imjuaniss/proyecto-melika
- Registro de cambios: CHANGELOG.md
- Documentación funcional adicional: docs/Frontend_Documentation.md y docs/Requerimientos_No_Funcionales.md
- Normativa clínica de referencia: Resolución 1995 de 1999 y Ley 2015 de 2020 (República de Colombia).

— Fin del Manual Técnico de MELIKA —